

RaiseError and @@ERROR function in SQL Server with Example

In this article, I am going to discuss the **RaiseError and @@ERROR Function in SQL Server** with Example. Please read our previous article where we discussed the [Basics Concepts of Exception Handling in SQL Server](#) with examples.

RaiseError Function in SQL Server

The system-defined **Raiserror()** function returns an error message back to the calling application. The RaiseError System defined Function in SQL Server takes 3 parameters as shown below.

RAISERROR('Error Message', ErrorSeverity, ErrorState)

1. **Error Message:** The custom error message that you want to display whenever the exception is raised.
2. **Error Severity:** When we are returning any custom errors in SQL Server, we need to set the ErrorSeverity level as 16, which indicates this is a general error and this error can be corrected by the user. In our example, the error can be corrected by the user by giving a nonzero value for the second parameter.
3. **Error State:** The ErrorState is also an integer value between 1 and 255. The RAISERROR() function can only generate custom errors if you set the Error State value between 1 to 127.

@@Error System Function in SQL Server:

In SQL Server 2000, in order to detect errors, we use the **@@Error** system function. The **@@Error** system function returns a **NON-ZERO** value if there is an error, otherwise, **ZERO** indicates that the previous SQL statement was executed without any error.

Example: RaiseError and @@ERROR Function in SQL Server

We are going to use the following Product and ProductSales table to understand how to handle errors in SQL Server using RaiseError and @@ERROR System-Defined Functions.

Product				ProductSales		
ProductId	Name	Price	QuantityAvailable	ProductSalesId	ProductId	QuantitySold
101	Laptop	1234	100	1	101	5
102	Desktop	3456	50	2	102	7
103	Tablet	5678	35	3	103	5
104	Mobile	7890	25	4	104	7

Please use the below SQL Script to create and populate the Product and ProductSales table with sample data.

```
-- Create Product table
```

```

CREATE TABLE Product
(
    ProductId INT PRIMARY KEY,
    Name VARCHAR(50),
    Price INT,
    QuantityAvailable INT
)
GO
-- Populate the Product Table with some test data
INSERT INTO Product VALUES(101, 'Laptop', 1234, 100)
INSERT INTO Product VALUES(102, 'Desktop', 3456, 50)
INSERT INTO Product VALUES(103, 'Tablet', 5678, 35)
INSERT INTO Product VALUES(104, 'Mobile', 7890, 25)
GO
-- Create ProductSales table
CREATE TABLE ProductSales
(
    ProductSalesId INT PRIMARY KEY,
    ProductId INT,
    QuantitySold INT
)
GO
-- Populate the ProductSales Table with some test data
INSERT INTO ProductSales VALUES(1, 101, 5)
INSERT INTO ProductSales VALUES(2, 102, 7)
INSERT INTO ProductSales VALUES(3, 103, 5)
INSERT INTO ProductSales VALUES(4, 104, 7)
Go

```

Create the following stored procedure for product sales.

The following stored procedure accepts 2 parameters – ProductID and QuantityToSell. The ProductID parameter specifies the product that we want to sell and the QuantityToSell parameter specifies the quantity that we want to sell. In the below procedure, if enough stock is not available then we are raising a custom exception by using the Raiserror statement.

```

CREATE PROCEDURE spSellProduct
@ProductID INT,
@QuantityToSell INT
AS
BEGIN
    -- First we need to Check the stock available for the product we want to sell
    DECLARE @StockAvailable INT
    SELECT @StockAvailable = QuantityAvailable FROM Product WHERE ProductId = @Pr
    -- We need to throw an error to the calling application
    -- if the stock is less than the quantity we want to sell
    IF(@StockAvailable < @QuantityToSell)
    BEGIN
        Raiserror('Enough Stock is not available', 16, 1)
    END

```

```

-- If enough stock is available
ELSE
BEGIN
-- We need to start the transaction
BEGIN TRANSACTION
-- First we need to reduce the quantity available
UPDATE Product SET QuantityAvailable = (QuantityAvailable - @QuantityToSell)
WHERE ProductID = @ProductID
-- Then Calculate MAX ProductSalesId
DECLARE @MaxProductSalesId INT
SELECT @MaxProductSalesId = CASE
WHEN MAX(ProductSalesId) IS NULL THEN 0
ELSE MAX(ProductSalesId)
END
FROM ProductSales
-- Increment @MaxProductSalesId by 1, so we don't get a primary key violation
Set @MaxProductSalesId = @MaxProductSalesId + 1
-- We need to insert the quantity sold into the ProductSales table
INSERT INTO ProductSales(ProductSalesId, ProductId, QuantitySold)
VALUES(@MaxProductSalesId, @ProductId, @QuantityToSell)
COMMIT TRANSACTION
End
END

```

The problem with the above-stored procedure is that the transaction is always going to be committed even though there is an error somewhere between updating the Product table and inserting data into the ProductSales table.

The main purpose of wrapping these 2 statements (Update Product Statement and Insert into ProductSales statement) in a transaction is to ensure that, both of these statements are treated as a single unit. For example, if we have an error when executing the second statement, then the first statement should be rolled back.

Let us modify the stored procedure to use the @@ERROR function to check if there any error occurred. If no error occurred then we are committing the transaction else we roll backing the transaction.

```

ALTER PROCEDURE spSellProduct
@ProductID INT,
@QuantityToSell INT
AS
BEGIN
-- First we need to Check the stock available for the product we want to sell
DECLARE @StockAvailable INT
SELECT @StockAvailable = QuantityAvailable FROM Product WHERE ProductId = @ProductID
-- We need to throw an error to the calling application
-- if the stock is less than the quantity we want to sell
IF(@StockAvailable < @QuantityToSell)
BEGIN
Raiserror('Enough Stock is not available',16,1)
END
-- If enough stock is available

```

```

ELSE
BEGIN
    -- We need to start the transaction
    BEGIN TRANSACTION
    -- First we need to reduce the quantity available
    UPDATE Product SET QuantityAvailable = (QuantityAvailable - @QuantityToSell)
    WHERE ProductID = @ProductID

    -- Calculate MAX ProductSalesId
    DECLARE @MaxProductSalesId INT
    SELECT @MaxProductSalesId = CASE
        WHEN MAX(ProductSalesId) IS NULL THEN 0
        ELSE MAX(ProductSalesId)
    END
    FROM ProductSales
    -- Increment @MaxProductSalesId by 1, so we don't get a primary key violation
    Set @MaxProductSalesId = @MaxProductSalesId + 1
    -- We need to insert the quantity sold into the ProductSales table
    INSERT INTO ProductSales(ProductSalesId, ProductId, QuantitySold)
    VALUES(@MaxProductSalesId, @ProductId, @QuantityToSell)
    -- The @@Error returns a NON-ZERO value if there is an error, otherwise it v
    ZERO,
    -- indicating that the previous SQL statement encountered no errors
    IF(@@ERROR <> 0)
    BEGIN
        ROLLBACK TRANSACTION
        PRINT 'Rolled Back the Transaction'
    END
    ELSE
    BEGIN
        COMMIT TRANSACTION
        PRINT 'Committed the Transaction'
    END
End
END

```

In the above procedure, if you comment the line (**Set @MaxProductSalesId = @MaxProductSalesId + 1**), and then execute the stored procedure there will be a primary key violation error when trying to insert into the ProductSales table as a result of which the entire transaction will be rolled back.

Note: The @@ERROR is cleared and reset on each statement execution. Check it immediately following the statement being verified, or save it to a local variable that can be checked later.

In the **Product** table, we already have a record with **ProductID = 4**. So the insert statement causes a primary key violation error. The @@ERROR retains the error number, as we are checking for it immediately after the statement that causes the error.

```

INSERT INTO Product values(4, 'Mobile Phone', 1500, 100)
IF(@@ERROR <> 0)
    PRINT 'Error Occurred'
ELSE

```

```
PRINT 'No Errors'
```

On the other hand, when you execute the code below, you will get the message '**No Errors**'. This is because the @@ERROR is cleared and reset on each statement execution.

```
INSERT INTO Product values(4, 'Mobile Phone', 1500, 100)
-- At this point the @@ERROR will have a NON ZERO value
SELECT * FROM Product
-- At this point the @@ERROR reset to ZERO, because the
-- select statement successfully executed
IF(@@ERROR <> 0)
    PRINT 'Error Occurred'
ELSE
    PRINT 'No Errors'
```

In the below example, we are storing the value of the @@Error function to a local variable, which is used later.

```
DECLARE @Error INT
INSERT INTO Product VALUES(4, 'Mobile Phone', 1500, 100)
Set @Error = @@ERROR
SELECT * FROM Product
IF(@Error <> 0)
    PRINT 'Error Occurred'
ELSE
    PRINT 'No Errors'
```

In the next article, I am going to discuss [how to raise errors explicitly in SQL Server](#), and also we will discuss the different options that we can use with the Raiserror function. Here, in this article, I try to explain the **RaiseError** and **@@Error Function in SQL Server** along with @@ERROR with Examples. I hope you enjoy this RaiseError in SQL Server with Examples article.